

User Manual

Compact Weather station communication protocol

ModBus-RTU V1.07M

CHENGDU SENTEC TECHNOLOGY CO., LTD

www.cdsentec.com

Compact Weather station communication protocol

ModBus-RTU V1.07M

Serial Port Configuration

Start Bit	1 bit
Data Bits	8 bit
Parity	EVEN
Stop Bits	1 bit
Baud Rate	9600 Baud

Communication mode

Communication mode:RS485 or RS232, default mode:**RS485**.

Communication Protocol

MODBUS Protocol - RTU Mode.

Protocol Description

MODBUS protocol defines a simple protocol data unit(PDU) independent from basic communication layer.

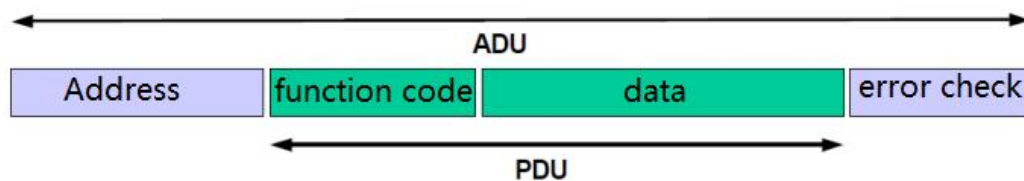


Figure 2. Commonly used MODBUS frame

MODBUS has two transmission mode:RTU and ASCII.

Our sensor adopts RTU mode.

1. RTU transmission mode

When controllers are setup to communicate on a Modbus network using RTU (Remote Terminal Unit) mode, each eight-bit byte in a message contains two four-bit hexadecimal characters. The main advantage of this mode is that its greater character density allows better data throughput than ASCII for the same baud rate. Each message must be transmitted in a continuous stream.

- **RTU Mode serial bits**



- **Modbus RTU message frame**

child node address	function code	data	CRC	
one bytes	one bytes	0~252 bytes	two bytes	
			CRC low	CRC high

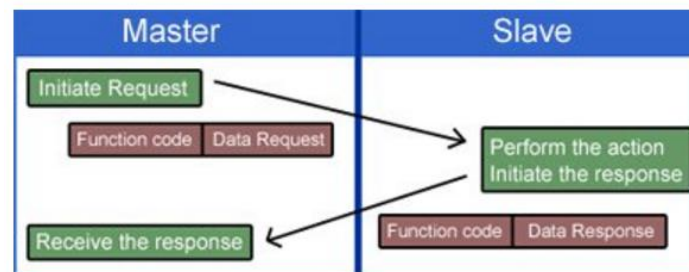
● CRC Check

RTU Mode has Cyclical Redundancy Checking(CRC) on all content of message, no matter if there is an odd-even check or not.

CRC check code is a 16 bits value composed by two 8 bits value and added as tail of message. After calculation, lower byte first then high byte. CRC higher byte is the last byte of message.

The CRC check code is calculated by sender. Receiver will recalculate CRC check code and compare it with CRC code received, if they are different, then there is an error happen during transmission.

MODBUS communication Mode



Data Coding

MODBUS use "big-Endian" to indicate address and data, which means when there is several bytes be sent out, the most significant bit is sent and received first.

Eg.

Register Size	Value
16bit	0x1234

The first byte is 0x12, then 0x34

1. Protocol of Device

➤ Function Code Supported

Function code type	Length	Function Code (HEX)	Description
Data access	16 bit	03	Read data from internal register
Data access	16 bit	10	Write data to multiple register

➤ Error Code Supported

Error code	Description
01	Function Code Error
02	Register Address Error
03	Register Value error
06	Device Busy

➤ Internal Registers Description:

Register	Length	Data Type	Definition	Range
Register 1	16 bit	16 bit int	Device State	0xA000~0xA03F
Register 2	16 bit	16 bit int	Wind Direction	0 - 359 °
Register 3	16 bit	32 bit float	Wind Speed	0 - +60 m/s
Register 4	16 bit			
Register 5	16 bit	32 bit float	Temperature	-40 - +120 °C
Register 6	16 bit			
Register 7	16 bit	32 bit float	Humidity	0 - 100 %
Register 8	16 bit			
Register 9	16 bit	32 bit float	Air Pressure	150 - 1100 hPa
Register 10	16 bit			
Register 11	16 bit	16 bit int	Compass Heading	0 - 359 °
Register 12	16 bit	16 bit int	Precipitation Type	0x0000 ~ 0x000F
Register 13	16 bit	32 bit float	Precipitation Intensity	Single-precision
Register 14	16 bit			
Register 15	16 bit	32 bit float	Accumulated Precipitation	Single-precision
Register 16	16 bit			
Register 17	16 bit	16 bit int	Intensity Unit	Refer to appendix
Register 18	16 bit	16 bit int	GPS Status	0: Positioned 1: No Positioned
Register 19	16 bit	32 bit float	GPS Speed	Km/h
Register 20	16 bit			
Register 21	16 bit	16 bit int	GPS Heading	0 - 359 °
Register 22	16 bit	32 bit float	Longitude	East: positive West: negative
Register 23	16 bit			
Register 24	16 bit	32 bit float	Latitude	North: positive South: negative
Register 25	16 bit			
Register 26	16 bit	32 bit float	Dust concentration	ug/m ³
Register 27	16 bit			
Register 28	16 bit	32 bit float	Visibility	m
Register 29	16 bit			
Register 30	16 bit	32 bit float	Radiation Illuminance	Lux
Register 31	16 bit			
Register 32	16 bit	32 bit float	Preserved for internal testing	
Register 33	16 bit			
Register 34	16 bit	32 bit float	Radiation Power	W/m ²
Register 35	16 bit			
Register 36	16 bit	16 bit	Compass Corrected Wind Direction	0 ~ 359 °
Register 37	16 bit	32 bit float	Altitude	m
Register 38	16 bit			

Note: Starting address of registers is from zero, E.g. address of register 1 is 0x0000

➤ 32 bit float type format

D3	D2	D1	D0
Higher byte	Middle byte 1	Middle byte 2	Lower byte

➤ Format of data stored in register

Definition	Register	Bit	Byte position
Wind speed	Register 2-higher byte	8 bit	D1
	Register 2-lower byte	8 bit	D0
	Register 3-higher byte	8 bit	D3
	Register 3-lower byte	8 bit	D2

➤ Function code(0x03) description - read holding register

A remote device can use function code to read data of holding register. The request PDU specifies starting address and quantity of registers. Register address from zero, therefore, the address register 1-3 corresponds to address 0-2.

Response packet from each register is divided into two bytes in binary format. The first byte is higher bits, the second byte is lower bits.

Request

Function Code	1 byte	0x03
Starting Address	2 bytes	0x0000 ~ 0x0025
Register Quantity	2 bytes	1 ~ 38

Response

Function Code	1 byte	0x03
Bytes Quantity	2 bytes	N*2
Register Data	N*2 bytes	

Note: N is quantity of registers

Error response

Error Code	1 byte	0x83
Exception Code	1 byte	01 or 02 or 03 or 06

Communication Example:

Read 38 internal registers

Communication Example:

➤ Request:

(HEX) 01 03 00 00 00 26 C4 10

➤ Response:

(HEX)

01 03 4C A7 FF 00 38 C2 8F 3C F5 33 33 41 E7 66 66 42 74 3E
14 44 68 00 0C 00 01 66 66 42 56 00 00 42 F7 02 01 00 01 99 9A
41 21 01 43 32 46 41 25 A5 9C 42 CF 00 00 42 70 00 00 42 C8
40 00 45 9C 00 00 43 48 39 9A 45 59 00 44 33 33 42 49 EF E8

Request		Response		Interpretation
Name	HEX	Name	HEX	
Instrument address (Default address 01)	01	Instrument address	01	
Function code (Read data from internal register)	03	Function code	03	
Starting address higher byte	00	Bytes Quantity	4C	
Starting address lower byte	00	Register 1 data higher byte	A7	Device State (Oxa7ff)
Quantity of register will be read- higher byte	00	Register 1 data lower byte	FF	
Quantity of register will be read- lower byte	26	Register 2 data higher byte	00	Wind Direction Int type (56)
CRC higher byte	C4	Register 2 data lower byte	38	
CRC lower byte	10	Register 3 data higher byte	C2	Wind Speed Float type (0.03)
		Register 3 data lower byte	8F	
		Register 4 data higher byte	3C	
		Register 4 data lower byte	F5	
		Register 5 data higher byte	33	Temperature (28.9)
		Register 5 data lower byte	33	
		Register 6 data higher byte	41	
		Register 6 data lower byte	E7	
		Register 7 data higher byte	66	Humidity (61.1)
		Register 7 data lower byte	66	
		Register 8 data higher byte	42	
		Register 8 data lower byte	74	
		Register 9 data higher byte	3E	Air Pressure (929.0)
		Register 9 data lower byte	14	
		Register 10 data higher byte	44	
		Register 10 data lower byte	68	
		Register 11 data higher byte	00	Compass Heading (12)
		Register 11 data lower byte	0C	
		Register 12 data higher byte	00	Precipitation Type (rain)
		Register 12 data lower byte	01	
		Register 13 data higher byte	66	Precipitation Intensity (53.6)
		Register 13 data lower byte	66	
		Register 14 data higher byte	42	
		Register 14 data lower byte	56	

	Register 15 data higher byte	00	Accumulated Precipitation
	Register 15 data lower byte	00	
	Register 16 data higher byte	42	
	Register 16 data lower byte	F7	
	Register 17 data higher byte	02	Intensity unit (mm/h)
	Register 17 data lower byte	01	
	Register 18 data higher byte	00	GPS Positioning Status (1)
	Register 18 data lower byte	01	
	Register 19 data higher byte	99	Traveling Speed (10.1)
	Register 19 data lower byte	9A	
	Register 20 data higher byte	41	
	Register 20 data lower byte	21	
	Register 21 data higher byte	01	Traveling Direction (323)
	Register 21 data lower byte	43	
	Register 22 data higher byte	32	Longitude (10.324774)
	Register 22 data lower byte	46	
	Register 23 data higher byte	41	
	Register 23 data lower byte	25	
	Register 24 data higher byte	A5	Latitude (103.823456)
	Register 24 data lower byte	9C	
	Register 25 data higher byte	42	
	Register 25 data lower byte	CF	
	Register 26 data higher byte	00	Dust Concentration (60.0)
	Register 26 data lower byte	00	
	Register 27 data higher byte	42	
	Register 27 data lower byte	70	
	Register 28 data higher byte	00	Visibility (100)
	Register 28 data lower byte	00	
	Register 29 data higher byte	42	
	Register 29 data lower byte	C8	
	Register 30 data higher byte	40	Illuminance (5000)
	Register 30 data lower byte	00	
	Register 31 data higher byte	45	
	Register 31 data lower byte	9C	
	Register 32 data higher byte	00	Reserved for internal testing (200)
	Register 32 data lower byte	00	
	Register 33 data higher byte	43	
	Register 33 data lower byte	48	
	Register 34 data higher byte	39	Solar Radiation (3475.6)
	Register 34 data lower byte	9A	
	Register 35 data higher byte	45	
	Register 35 data lower byte	59	

		Register 36 data higher byte	00	Compass Corrected Wind Direction
		Register 36 data lower byte	44	
		Register 37 data higher byte	33	Altitude (50.3)
		Register 37 data lower byte	33	
		Register 38 data higher byte	42	
		Register 38 data lower byte	49	

➤ Int type

Take wind direction for example

D1	D0
Register 2 higher byte	Register 2 lower byte
00	38
higher byte	lower byte

Transformed as int type, value is **0x0038 => 56°**

➤ Float type(IEEE754 Standard)

Take temperature for example

D3	D2	D1	D0
Register 6 higher byte	Register 6 lower byte	Register 5 higher byte	Register 5 lower byte
41	E7	33	33
higher byte	middle byte1	middle byte2	lower byte

Convert to float type, value is **0x41E73333 => 28.9 °C**

➤ Device State(Device State Sheet)

BIT 15	BIT 14	BIT 13	BIT 12	BIT 11	BIT 10	BIT 9	BIT 8
1	0	1	0	0	0	1/0	1/0
					Altitude	Visibility	Illuminance
BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
1/0	1/0	1/0	1/0	1/0	1/0	1/0	1/0
Radiation Power Daily Radiation	Dust Concentration	GPS Status GPS Speed GPS Heading Longitude Latitude	Precipitation State Precipitation Intensity Accumulatd Precipitation Precipitation Unit	Compass Heading Compass Corrected Wind Direction	Air Pressure	Wind Speed Wind Direction	Temperature Humidity

Note: Only when status bit is "1", corresponding data is valid, otherwise it's invalid.

➤ Precipitation Intensity Unit (HEX)

higher 8-bits	lower 8-bits
0 : mm/s	reserved
1 : mm/min	
2 : mm/hour	

➤ Precipitation State (HEX)

BIT 15	BIT 14	BIT 13	BIT 12	BIT 11	BIT 10	BIT 9	BIT 8
--------	--------	--------	--------	--------	--------	-------	-------

0	0	0	0	0	0	0	0
BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
0	0	0	0	0/1	solid 0/1	sno w 0/1	rain 0/1

Note: Only the last 3-bits are used, "1" at corresponding bit means valid, otherwise it's invalid.

Clear accumulated precipitation Command(fixed string):

- Request:
(HEX) 01 10 00 0F 00 02 04 00 00 00 00 B3 EF
- Response:
(HEX) 01 10 00 0F 00 02 71 CB

Parameters Setting Commands

Following parameters can be set by user, such as communication address or baud rate.

Instruction Set

Instruction 1 (Enter Setting Mode)

Send

(ASCII) >*\r\n

(HEX) 3E 2A 0D 0A

Return

(ASCII) \n>CONFIGURE MODE\r\n

(HEX) 0A 3E 43 4F 4E 46 49 47 55 52 45 20 4D 4F 44 45 0D 0A

Instruction 2 (Configure Serial Port)

Send

(ASCII) >CUS 9600 8-N-1\r\n

(HEX) 3E 43 55 53 20 39 36 30 30 20 38 2D 4E 2D 31 0D 0A

Note: Characters after "CUS" is the value serial port will be set. If it leave empty, this instruction will become an inquiry

instruction(>CUS\r\n), instrument will return corresponding data:

\n>COM USART SET : 9600 N-8-1\r\n

Return

(ASCII) >CMD IS SET\r\n

(HEX) 3E 43 4D 44 20 49 53 20 53 45 54 0D 0A

Instruction 3 (set address)

Send

(ASCII) >ID 2\r\n

(HEX) 3E 49 44 20 32 0D 0A

Note: The character "2" after "ID" is the value that address will be set, it must be decimal value (1-255), otherwise this

invalid instruction will become an inquiry instruction(>ID\r\n), instrument will return corresponding data:

>ID(HEX) : 02\r\n

Return

(ASCII) >CMD IS SET\r\n

(HEX) 3E 43 4D 44 20 49 53 20 53 45 54 0D 0A

Instruction 4 (Reset)

Send

(ASCII) >RESET\r\n

(HEX) 3E 52 45 53 45 54 0D 0A

Note: This instruction will soft reset instrument.

Instruction 5 (Manually Exit Setting Mode)

Send

(ASCII) >!\r\n

(HEX) 3E 21 0D 0A

Return

(ASCII) \n>NORMAL MODE\r\n

(HEX) 0A 3E 4E 4F 52 4D 41 4C 20 4D 4F 44 45 0D 0A

Note: This instruction will make instrument exit setting mode.

Operating Procedures

1. Set Communication Address

Steps: "Instruction 1" → "Instruction 3" → "Instruction 5" → "Instruction 4"

2. Set Serial Port Parameters

Steps: "Instruction 1" → "Instruction 2" → "Instruction 5" → "Instruction 4"

Remind:

There are two "space" characters in string "CUS 9600 8-N-1",
8-N-1 is connected with two characters "-", without "space"

Baud Rate	Data Bit	Checksum	Stop Bit
2400~115200	7 bits(supported by some protocol) 8 bits	N:None	1
		O:Odd	2
		E:Even	

1. User have to input instruction 1 to make instrument enter setting mode before input instruction 2/3, instrument will automatically exit setting mode without any input within 15 seconds.
2. Input instruction 4 to soft reset instrument after changing settings, so that changes will take affect.
3. Characters "\r\n" is CRLF Carriage-Return Line-Feed, corresponding to HEX (0x0D,0x0A)
4. After inputing "Instruction 5" wait for 2 seconds before inputing "Instruction 4", the instrument will take 2 seconds to activate setting.

I .Appendix LRC Verification

A. Using a C language function code to generate LRC values.

The function code uses 2 independent variables:

unsigned char *auchMsg; // To generate the LRC value, point the pointer to the buffer containing the binary data

unsigned short usDataLen; //Number of bytes in the buffer.

//This function returns LRC as a type“unsigned char” .

// RC check code generation

static unsigned char LRCCheck(auchMsg, usDataLen)

unsigned char *auchMsg; /* calculating by information byte LRC*/

unsigned short usDataLen; /*calculating by information byteLRC*/

```
{
    unsigned char uchLRC = 0 ; /*Initializing LRC characters */
    while (usDataLen--) /*through the data buffer*/
        uchLRC += *auchMsg++; /*Add buffer byte Buffer byte no carry*/
    return ((unsigned char)(~((char)uchLRC))) ; /*reture to Binary complement*/
}
```

II .Appendix transform HEX to float data

Use C language's subfunction to transform 4 bytes(HEX) as float data(C language)
union

```
{  
float TestData_Float;  
unsigned char TestArray[4];  
}TData;
```

Analysis example:

D3	D2	D1	D0
Higher byte of register 2	Lower byte of register 2	Higher byte of register 1	Lower byte of register 1
40	AC	19	DF
Higher byte	Middle byte 1	Middle byte 2	Lower byte

After transformed to float data, value: 5.378

Subfunction:

```
float Tempfloat;  
TData.TestArray [3]= 0x40; //input higher byte  
TData.TestArray [2]= 0xac; //  
TData.TestArray [1]= 0x19; //  
TData.TestArray [0]= 0xdf; //input lower byte  
Tempfloat = TData.TestData_Float; //return result 5.378
```